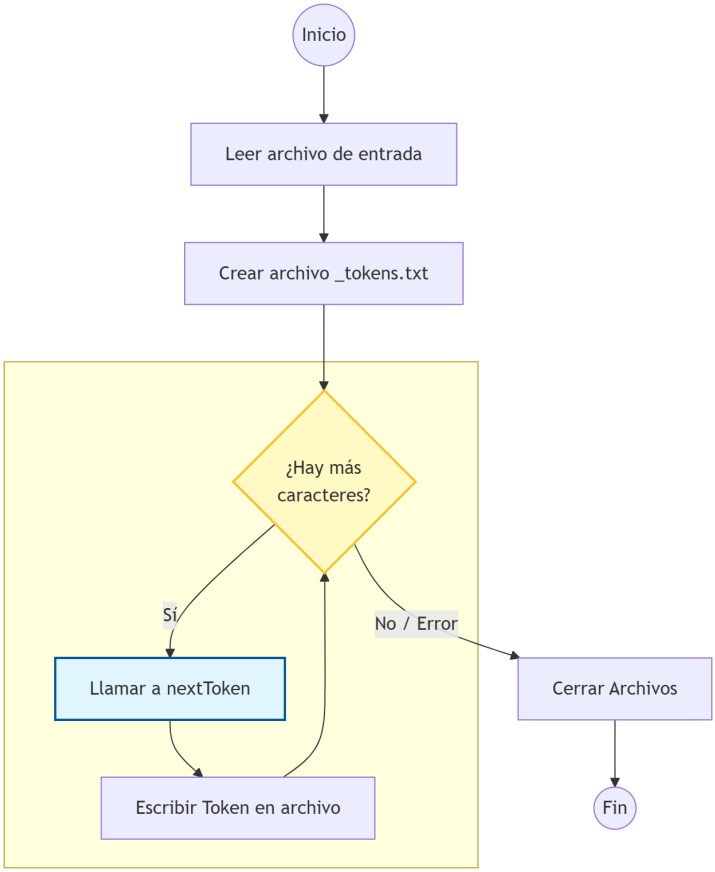
Evaluación Continua 1 - Prueba 1

El presente laboratorio detalla la implementación de un Analizador Léxico (Scanner) de una sola pasada (one-pass), diseñado en C++ siguiendo un esquema de diseño modular para la fase inicial de un compilador. La arquitectura del sistema se fundamenta en la separación de responsabilidades mediante tres componentes críticos:

- Unidad de Control (main.cpp): Actúa como el motor de ejecución que gestiona el flujo de E/S (entrada/salida), encargándose de la carga del código fuente en memoria y la invocación iterativa del proceso de escaneo.
- Analizador de Lexemas (scanner.cpp): Implementa la lógica de reconocimiento mediante el método nextToken(), el cual utiliza punteros de lectura (first y current) para recorrer la cadena de entrada. Este componente destaca por su capacidad de agrupar secuencias de caracteres mediante un bucle de consumo iterativo para identificar constantes numéricas y distinguir operadores aritméticos de caracteres inválidos.
- Representación de Símbolos (token.cpp): Define la estructura de datos que encapsula cada unidad léxica identificada, almacenando tanto su categoría (tipo de token) como su valor textual (lexema), facilitando así la posterior fase de análisis sintáctico.



Enunciados

1. Implementar el reconocimiento de comentarios de una sola línea utilizando el símbolo `#`. El scanner debe ignorar el contenido hasta el final de la línea.

2. Extender la lógica para reconocer llamadas a funciones matemáticas predefinidas, específicamente: `SIN`, `COS` y `LOG`.

3. Modificar el reconocimiento de constantes para permitir la entrada de números en formato binario. Se deben definir el token `BIN`.

4. Agregar el soporte léxico para los siguientes elementos, creando sus respectivos tokens:

   - **Booleanos:** Literales `True` y `False`.
   - **Operadores Relacionales:** `<`, `>`, `==` y `!=`.

5. El escáner implementado debe ser capaz de procesar y categorizar correctamente los siguientes ejemplos:

   - 10 + 20 # Suma de valores con comentario lateral
   - # SIN(90) no debe generar tokens
   - COS(0) * LOG(100)
   - 0b11 + 0b01
   - SIN(LOG(100)) + COS(0b101)
   - True + True
   - 0b10101101 != 0b111
   - 10 == 10
   - (2+2) <5 + COS(15)
   - SIN(90) >0b1 # Mezcla de tipos y funciones